

---

# Calculus for Machine Learning

## Refresher Module 3, Part 2 — Integration & the PID Controller

accumulation & area · numerical integration & drift · Euler's method · PID: the I, tuning, windup, and the D

---

Computing, Robotics & Bionics

3<sup>rd</sup>-year Bioengineering — companion to A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This is the second half of the third of four refresher modules. **Part 1**, taught earlier alongside the machine-learning units, developed the derivative, the gradient and Jacobian, and gradient descent. **Part 2** (this document) treats integration as accumulation and area, the Fundamental Theorem that ties it back to differentiation, numerical integration and the drift it builds from sensor errors, and Euler's method — the integral behind every simulation — then cashes it out in a PID controller: the **I** that must drive the steady-state error to zero, tuning predicted from a characteristic equation, integral windup, and the **D** that damps a prosthetic elbow at the price of amplifying noise. It links this module directly to the companion Control Theory unit. It is **self-contained**: together with its companion Colab notebook — a bioengineering-adapted superset of Géron's integration material — it is your complete reference for this half. Each section ends with exercises.*

Prerequisites: Modules 1–2 (NumPy, linear algebra) and Module 3 Part 1 (the derivative). Software: a web browser and a Google account (everything runs in Google Colab; nothing to install).

## Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Integration: Accumulation and Area</b>         | <b>2</b>  |
| 1.1      | The Fundamental Theorem . . . . .                 | 2         |
| 1.2      | Numerical integration . . . . .                   | 2         |
| 1.3      | Integration accumulates errors: drift . . . . .   | 3         |
| <b>2</b> | <b>Simulating a System Is Integration</b>         | <b>5</b>  |
| <b>3</b> | <b>The Integral in Action: the “I” of PID</b>     | <b>6</b>  |
| 3.1      | Proportional control leaves an offset . . . . .   | 6         |
| 3.2      | Why the integral must remove it . . . . .         | 7         |
| 3.3      | Tuning from the characteristic equation . . . . . | 7         |
| 3.4      | Integral windup . . . . .                         | 8         |
| <b>4</b> | <b>The D Term: Damping, and Its Price</b>         | <b>9</b>  |
| 4.1      | Derivative kick . . . . .                         | 10        |
| 4.2      | The derivative amplifies noise . . . . .          | 10        |
| <b>5</b> | <b>The Lab, and Where This Leads in Géron</b>     | <b>11</b> |

## 1 Integration: Accumulation and Area

Differentiation measures a **rate**; integration measures **accumulation** — the total built up from a rate over time, equivalently the **area under a curve**. The definite integral is the limit of a **Riemann sum** of thin rectangles:

$$\int_a^b f(t) dt = \lim_{\Delta t \rightarrow 0} \sum_k f(t_k) \Delta t. \quad (1.1)$$

### 1.1 The Fundamental Theorem

Integration and differentiation are inverses: if  $F' = f$  then  $\int_a^b f(t) dt = F(b) - F(a)$ , and

$$\frac{d}{dx} \int_a^x f(t) dt = f(x). \quad (1.2)$$

So accumulating a rate recovers the quantity, and differentiating an accumulation recovers the rate — the two halves of this module are two sides of one coin. The second form, (1.2), will do real work later: it is what lets us prove that a PID controller's integral term *must* remove a steady-state error (Section 3.2), and what turns a controller with an integral in it into an ordinary differential equation we can solve (Section 3.3).

### 1.2 Numerical integration

When  $f$  is data, or has no closed-form antiderivative, approximate the area from samples  $f_k = f(t_k)$  on a grid of spacing  $h$ . Each rule replaces  $f$  on a strip by something simpler (Figure 1, left): a flat top (the **left** or **midpoint** rule), a straight line through both ends (the **trapezoidal rule**), or a parabola through three consecutive points (**Simpson's rule**, which needs an even number  $N$  of strips):

$$\int_a^b f dt \approx h \left( \frac{1}{2} f_0 + f_1 + \cdots + f_{N-1} + \frac{1}{2} f_N \right) \quad (\text{trapezoid}), \quad (1.3)$$

$$\int_a^b f dt \approx \frac{h}{3} \left( f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 2f_{N-2} + 4f_{N-1} + f_N \right) \quad (\text{Simpson}). \quad (1.4)$$

#### How fast the error shrinks

With strips of width  $h$ , the total error of each rule is

$$\underbrace{O(h)}_{\text{left, right}} \quad \underbrace{O(h^2)}_{\text{midpoint, trapezoid}} \quad \underbrace{O(h^4)}_{\text{Simpson}}.$$

Halving  $h$  therefore divides the error by 2, 4 or 16.

The reason is the Taylor bookkeeping of Part 1 (§4.2). On one strip, a flat top misses the slope and is wrong by  $O(h^2)$ ; there are  $N = (b-a)/h$  strips, so the total is  $O(h)$ . The trapezoid is exact for straight lines, so its per-strip error starts at the curvature term,  $O(h^3)$ , and the total is  $O(h^2)$ . A parabola is exact for quadratics, and by symmetry for cubics as well, so Simpson's per-pair error is  $O(h^5)$  and its total  $O(h^4)$  — the same symmetry bonus that made the central difference beat the forward one. Figure 1 (right) measures the three slopes.

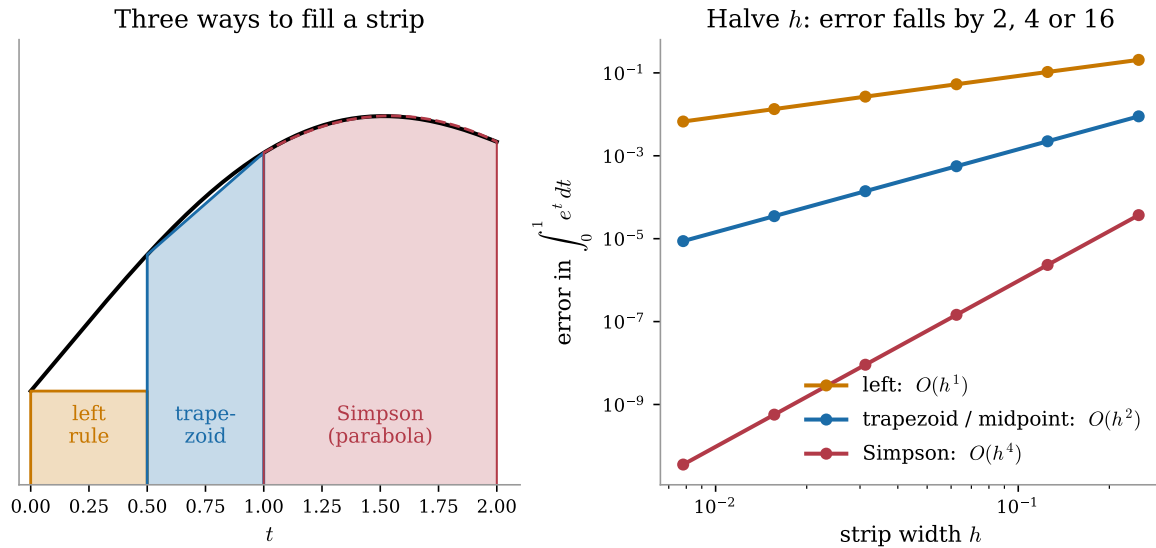


Figure 1: Left: what each rule puts on a strip (Simpson uses a pair of strips). Right: the error in  $\int_0^1 e^t dt$  against the strip width, on log-log axes, where an error  $O(h^p)$  is a straight line of slope  $p$ .

#### Same three samples, sixty times the accuracy

Estimate  $\int_0^1 e^t dt = e - 1 = 1.718282$  from  $t = 0, 0.5, 1$  ( $h = 0.5$ ). The trapezoid gives  $0.5 (\frac{1}{2} \cdot 1 + 1.648721 + \frac{1}{2} \cdot 2.718282) = 1.753931$ , an error of 0.0356. Simpson gives  $\frac{0.5}{3} (1 + 4 \cdot 1.648721 + 2.718282) = 1.718861$ , an error of 0.00058. Same data, same cost — only the weights changed.

**Two physiological examples.** Integrating a respiratory *flow* signal (L/s) over time gives the lung *volume* (L): the area under the flow curve. And in pharmacokinetics the **area under the concentration curve**, the AUC, is the standard measure of total drug exposure — computed from a handful of blood samples with exactly (1.3). You will do both in the lab.

### 1.3 Integration accumulates errors: drift

Integration adds up *everything* in the signal, including its errors. Suppose a flow sensor has a small **bias**  $\beta$  (it reads  $\beta$  L/s when nothing flows) and independent **noise** of size  $\sigma$  per sample. Then

- the bias contributes  $\int_0^t \beta d\tau = \beta t$ : the computed volume **drifts linearly**, without limit;
- the noise adds like a random walk, and its contribution grows like  $\sigma\sqrt{\Delta t}$  — slower, but also without limit.

A bias of 0.01 L/s is invisible on a flow trace; after a minute it has added 0.6 L to the volume, about one tidal volume (Figure 2, left). The usual fix uses physiology: over whole breaths air in equals air out, so the true mean flow is zero, and subtracting the measured mean before integrating removes the bias.

Integrate *twice* and it gets worse. A wearable IMU that tracks position from acceleration turns a bias  $\beta$  into a velocity error  $\beta t$  and a position error  $\frac{1}{2}\beta t^2$ : an accelerometer bias of 0.02 m/s<sup>2</sup> makes a perfectly still wrist “move” a metre in ten seconds (Figure 2, right). That is why inertial trackers need an outside reference — a camera, or a zero-velocity reset at every footstep.

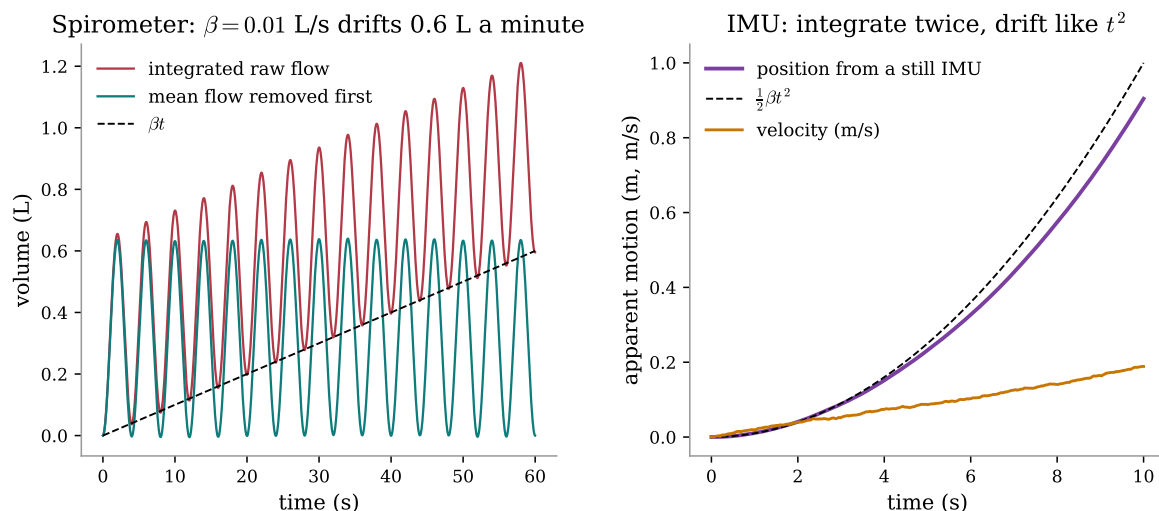


Figure 2: Left: a minute of breathing through a flow sensor with a 0.01 L/s bias. The raw integral climbs along  $\beta t$ ; removing the mean flow first removes the drift. Right: a stationary IMU with a 0.02 m/s<sup>2</sup> bias. Velocity drifts like  $t$ , position like  $t^2$ .

### Key idea

**Differentiation amplifies noise; integration accumulates bias.** Part 1 (§2) showed that a finite difference divides by  $h$  and so magnifies every error in the samples. An integral multiplies by  $h$  and sums, so independent errors partly cancel — but a constant error never does. Both facts return in the PID controller: the I term is what removes a constant error, and the D term is the one that amplifies noise (Section 4.2).

### A note on integrals in probability

Integration returns later as *area under a probability density*: the expectation  $\mathbb{E}[X] = \int x p(x) dx$  and the probability of an interval are integrals. Géron uses these ideas when discussing distributions and likelihoods; the mechanics are the same accumulation you meet here.

### Exercises

1. Approximate  $\int_0^1 x^2 dx$  with the trapezoidal rule using  $N = 4$  intervals, and compare to the exact  $1/3$ . Repeat with Simpson's rule, and explain why its answer is exact.
2. A rule's error is  $8.0 \times 10^{-3}$  with  $h = 0.1$  and  $5.0 \times 10^{-4}$  with  $h = 0.05$ . What is its order, and which rule could it be?
3. A spirometer has a flow bias of 0.005 L/s. What volume error does it accumulate over a 5-minute recording if the raw flow is integrated? How does the mean-removal fix rely on the recording containing whole breaths?
4. An IMU accelerometer has a bias of 0.01 m/s<sup>2</sup>. Estimate the apparent displacement after 30 s of double integration, and explain in one sentence why the error grows faster than with single integration.

## 2 Simulating a System Is Integration

A model such as  $\dot{x} = -kx$  (a drug eliminated at a rate proportional to what is left) says how fast  $x$  *changes*, not what  $x$  *is*. To get  $x(t)$  we integrate the rate,  $x(t) = x(0) + \int_0^t \dot{x} d\tau$ . **Euler’s method** does it one strip at a time with a left Riemann sum: for  $\dot{x} = f(x)$ ,

$$x_{k+1} = x_k + \Delta t f(x_k). \quad (2.1)$$

Every simulation in the lab — the line `x += dt*(-a*x + b*u)` — is exactly this, and Section 1.2 tells us two things about it at once. Being a left rule, Euler’s global error is  $O(\Delta t)$ : halve the step, halve the error. And it has a step-size limit. For  $\dot{x} = -\lambda x$  one step gives  $x_{k+1} = (1 - \lambda\Delta t)x_k$ , which shrinks only if  $|1 - \lambda\Delta t| < 1$ .

### The step-size law

Euler’s method on  $\dot{x} = -\lambda x$  with  $\lambda > 0$  decays, as the true solution does, if and only if

$$0 < \Delta t < \frac{2}{\lambda}.$$

Between  $1/\lambda$  and  $2/\lambda$  the factor  $1 - \lambda\Delta t$  is negative and the simulated solution flips sign every step; above  $2/\lambda$  it grows without bound (Figure 3). For a system  $\dot{\mathbf{x}} = -\mathbf{A}\mathbf{x}$  the same condition must hold for every eigenvalue of  $\mathbf{A}$ , and the largest sets the limit.

### Key idea

This is Part 1’s **learning-rate law** (§6.1), letter for letter. Gradient descent  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$  is Euler’s method applied to the “gradient flow”  $\dot{\mathbf{w}} = -\nabla L(\mathbf{w})$ , with the learning rate as the time step. Near a minimum  $\nabla L \approx \mathbf{H}(\mathbf{w} - \mathbf{w}^*)$ , so the eigenvalues of the Hessian play the role of  $\lambda$ , and  $\eta < 2/\lambda_{\max}$  is  $\Delta t < 2/\lambda$ . A step too large for the fastest mode explodes, whether you are training a network or simulating a pump.

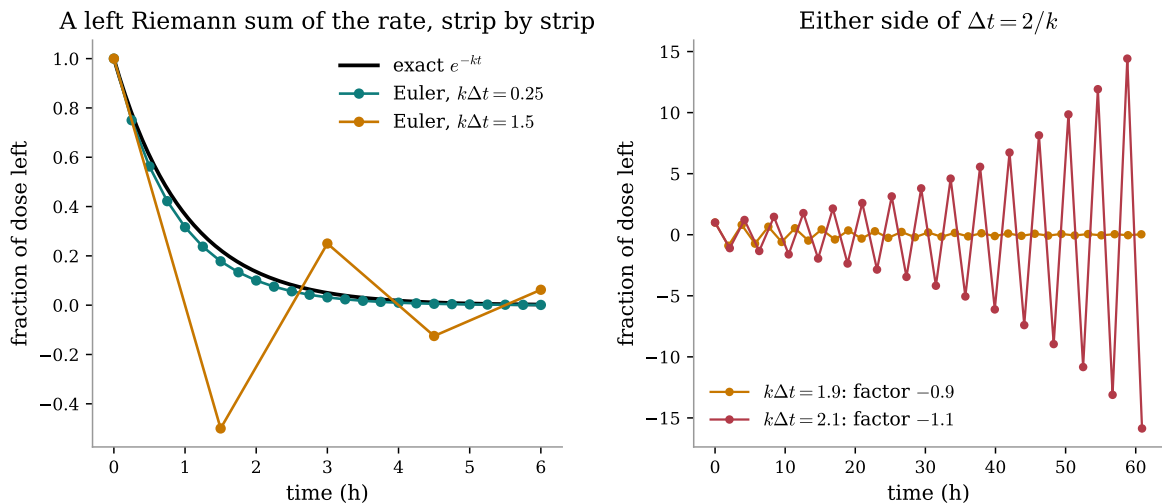


Figure 3: Euler’s method on drug elimination,  $\dot{x} = -kx$ . Left: a small step tracks the exact curve with error  $O(\Delta t)$ ; at  $k\Delta t = 1.5$  the factor is  $-0.5$  and the simulation rings, though it still decays. Right: either side of the limit  $k\Delta t = 2$ .

**Feedback shrinks the safe step.** Closing a control loop usually makes a system respond faster, which raises  $\lambda$ . Under proportional control  $u = K_p(r - x)$  the plant  $\dot{x} = -ax + bu$  becomes

$\dot{x} = -(a + bK_p)x + bK_pr$ ; with  $a = b = 1$  and  $K_p = 2$  the rate triples, and the largest stable step falls from 2 to 2/3. The lab shows it failing at  $\Delta t = 0.68$ .

#### Oscillators need care

For a lightly damped oscillator the eigenvalues are complex,  $\lambda = \zeta\omega \pm i\omega\sqrt{1 - \zeta^2}$ , and the condition  $|1 - \lambda\Delta t| < 1$  becomes  $\Delta t < 2\zeta/\omega$ : the less damping, the smaller the step, and with no damping at all plain Euler is never stable. That is why the elbow of Section 4 is simulated at  $\Delta t = 1$  ms, and why serious simulators use higher-order integrators such as Runge–Kutta (SciPy's `solve_ivp`).

#### Exercises

1. A drug is eliminated with  $k = 0.5 \text{ h}^{-1}$ . What is the largest Euler step that gives a decaying simulation? Which step gives a simulated curve that flips sign every hour?
2. For  $\dot{x} = -x$ ,  $x(0) = 1$ , Euler with  $\Delta t = 0.1$  gives  $x(1) \approx 0.3487$  against the exact  $e^{-1} = 0.3679$ . Predict the error with  $\Delta t = 0.05$ .
3. Write one step of gradient descent on  $L(w) = \frac{\lambda}{2}w^2$  and one Euler step of  $\dot{w} = -\lambda w$ . Show they are identical, and state which quantity plays the role of  $\Delta t$ .

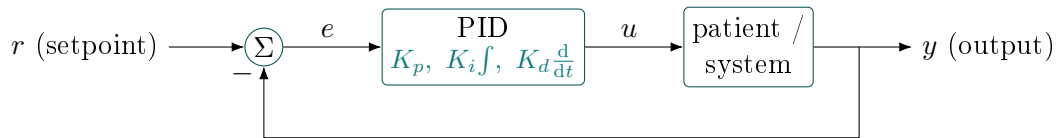
### 3 The Integral in Action: the “I” of PID

The clearest engineering use of integration — and the bridge to the Control unit — is the **integral term of a PID controller**. A controller drives a system to a target by acting on the error  $e(t) = \text{setpoint} - \text{measurement}$ :

$$u(t) = \underbrace{K_p e(t)}_P + \underbrace{K_i \int_0^t e(\tau) d\tau}_I + \underbrace{K_d \frac{de(t)}{dt}}_D. \quad (3.1)$$

In code, the integral is a running Riemann sum — an accumulator updated each step — and the derivative is a backward difference, the finite difference of Part 1 (§2) with its  $O(\Delta t)$  error and its sensitivity to noise:

$$\text{integral} += e\_k * dt, \quad u_k = K_p e_k + K_i \text{integral} + K_d \frac{e_k - e_{k-1}}{\Delta t}. \quad (3.2)$$



Throughout this section the system is first order,  $\dot{x} = -ax + bu$  — think of a drug concentration under infusion, or a temperature under heating — with a constant setpoint  $r$ .

#### 3.1 Proportional control leaves an offset

With  $u = K_p(r - x)$  the steady state,  $\dot{x} = 0$ , satisfies  $ax = bK_p(r - x)$ , so

$$x_{ss} = \frac{bK_p}{a + bK_p} r, \quad r - x_{ss} = \frac{a}{a + bK_p} r. \quad (3.3)$$

The offset is not necessarily small: with  $a = b = 1$  and  $K_p = 2$  it is a third of the setpoint. Raising  $K_p$  shrinks it but never removes it, because a proportional controller can only push

while an error remains — and the plant needs a push,  $u = ax/b$ , just to hold its level against its own decay.

#### Not every plant needs the I

The offset comes from the plant's own decay term  $-ax$ . A plant that integrates its input — a syringe filling a volume,  $\dot{x} = bu$  — needs no push to hold still, and P control alone drives it to the setpoint. The I term earns its keep on plants that leak, decay or carry a load.

### 3.2 Why the integral must remove it

“The integral keeps accumulating the residual error until it is zero” is an argument that can be made exact with (1.2).

#### The integral term forces zero steady-state error

Let  $I(t) = \int_0^t e \, d\tau$  be the integrator's state, so that  $\dot{I} = e$ . If the closed loop settles to any steady state, every signal stops changing —  $I$  included. Then  $\dot{I} = 0$ , and therefore  $e = 0$ .

The controller never needs to know how large the load or the decay is; it simply cannot sit still while any error remains. At steady state the push comes from the *stored* integral instead of from the error:  $u_{ss} = K_i I_{ss} = ar/b$ , so  $I_{ss} = ar/(bK_i)$  — a prediction the lab confirms to three digits. Note the theorem's one assumption: the loop must settle. Whether it does, and how fast, is the next question.

### 3.3 Tuning from the characteristic equation

The PI loop  $\dot{x} = -ax + bu$ ,  $u = K_p e + K_i I$ , is an integro-differential equation. Differentiate it once and (1.2) removes the integral: with  $r$  constant,  $\dot{e} = -\dot{x}$ , and

$$\ddot{x} + (a + bK_p)\dot{x} + bK_i x = bK_i r. \quad (3.4)$$

The PI loop is a **mass–spring–damper**:  $K_i$  is the spring and  $a + bK_p$  the damper. Trying  $e \propto e^{st}$  gives the characteristic equation

$$s^2 + (a + bK_p)s + bK_i = 0, \quad (3.5)$$

whose roots govern how the error dies away,  $e(t) \sim e^{-\sigma t}$  with  $\sigma = -\max \operatorname{Re}(s)$ . Two regimes follow.

- **Real roots**, for  $K_i < K_i^*$ : no ringing. The slower root sets the tail, and it speeds up as  $K_i$  grows.
- **Complex roots**, for  $K_i > K_i^*$ : the tail rings, and its decay rate is stuck at  $(a + bK_p)/2$  however large  $K_i$  becomes.

The boundary is a double root, **critical damping**, with damping ratio  $\zeta = (a + bK_p)/(2\sqrt{bK_i}) = 1$ :

$$K_i^* = \frac{(a + bK_p)^2}{4b}. \quad (3.6)$$

For  $a = b = 1$  and  $K_p = 2$  that is  $K_i^* = 2.25$ , predicted before a single simulation runs. Figure 4 checks it: the tail decay rate measured from each simulated response sits on the curve computed from the roots.

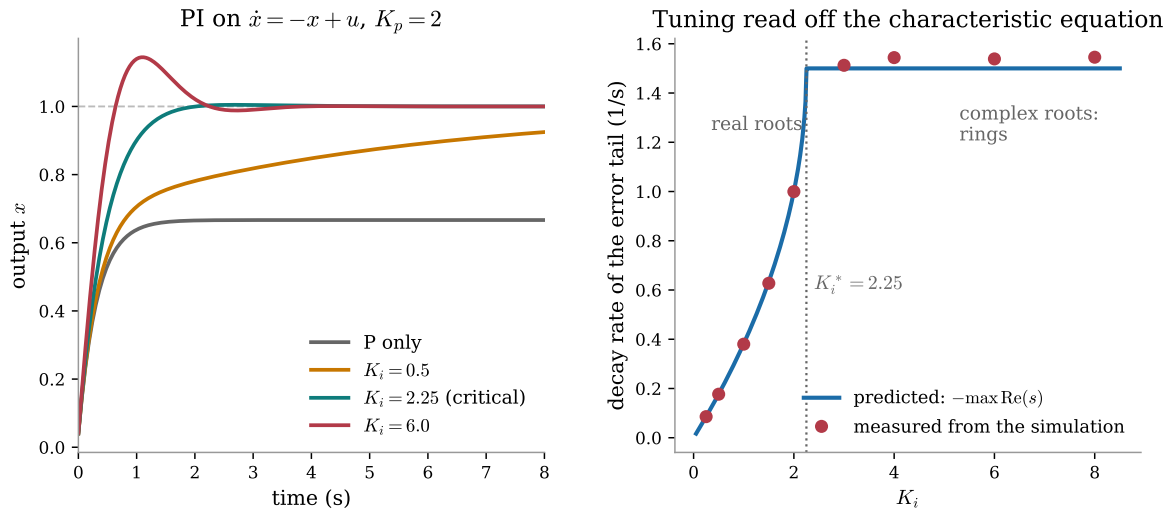


Figure 4: Left: step responses under P and PI control. P leaves the offset of (3.3); a small  $K_i$  removes it slowly;  $K_i^*$  removes it fastest without ringing; a large  $K_i$  rings. Right: the decay rate of the error's tail, predicted from the roots of (3.5) (line) and measured from simulations (dots).

#### What the roots do not predict

The roots fix the shape of the *tail*, not the first overshoot. The  $K_p$  term acts on  $\dot{e}$  and adds a zero to the response, which can carry it past the setpoint once even when both roots are real. With  $K_p = 2$  the effect is tiny; with  $K_p = 4$  overshoot above 1% already appears at  $K_i \approx 4.75$ , well below  $K_i^* = 6.25$ . Right at  $K_i^*$  the tail is  $te^{-\sigma t}$  rather than a pure exponential.

### 3.4 Integral windup

Real actuators have limits: an infusion pump cannot run backwards,  $u \geq 0$ , or faster than its maximum rate. While  $u$  is pinned at the limit the error persists, so the integral keeps growing — it **winds up** — with no effect on the plant. When the output finally reaches the setpoint, the swollen integral keeps pushing, and the output overshoots until the excess has been paid back by error of the opposite sign: by the area picture, the overshoot is the integral repaying what it stored (Figure 5).

The simplest cure is **conditional integration**: stop integrating while the actuator is saturated. In the lab, with the pump limited to 1.2 and  $K_i = 4$ , it cuts the peak from 1.178 to 1.014. **Clamping** the integrator to  $0 \leq I \leq u_{\max}/K_i$  also helps (peak 1.044), but still lets  $I$  climb above the value it finally needs.

#### Key idea

In the lab you simulate a first-order biomedical system under **P-only** and then **PI** control and watch the integral term erase the offset that P alone leaves behind; you predict the best  $K_i$  from (3.6) and measure it; and you add a pump limit and fix the windup it causes. That is calculus — accumulation — doing real control work, and it is the quantitative version of the qualitative story in the Control Theory unit.

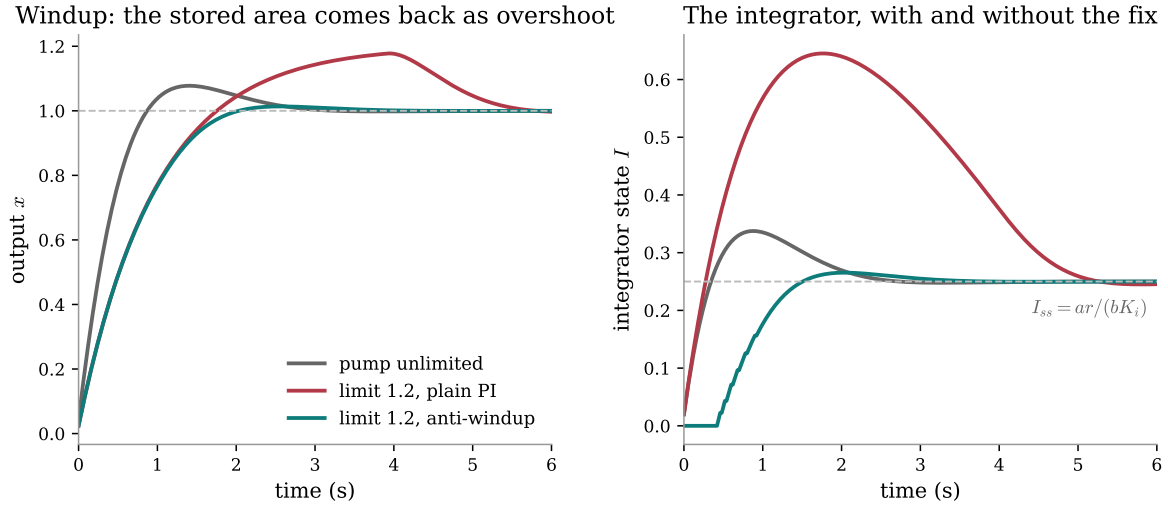


Figure 5: PI control ( $K_p = 2$ ,  $K_i = 4$ ) with a pump limited to 1.2 units; holding the setpoint needs 1. Left: the output. Right: the integrator state. Plain PI stores far more than the  $I_{ss} = 0.25$  it needs, and pays it back as overshoot; conditional integration stores almost nothing extra.

### Exercises

1. Explain, in two sentences, why a proportional-only controller leaves a steady-state offset and how the integral term removes it. Use (3.3) to compute the offset for  $a = b = 1$ ,  $K_p = 9$ .
2. Derive (3.4) from  $\dot{x} = -ax + bu$  and  $u = K_p e + K_i \int e$ .
3. For  $a = b = 1$  and  $K_p = 4$ , compute  $K_i^*$ , the fastest tail decay rate the loop can reach, and the roots of (3.5) at  $K_i = 10$ .
4. The theorem of Section 3.2 assumes the loop settles. Give an example, using (3.5), of gains for which it would not.
5. With windup, why must the output overshoot? Answer using the area under  $e(t)$ .

## 4 The D Term: Damping, and Its Price

A first-order plant does not need D. A **prosthetic elbow** does, because it has inertia. With joint angle  $\theta$ , motor torque  $u$ , inertia  $J$ , a little joint friction  $c$  and a load torque  $\tau_L$  from whatever the hand is holding,

$$J\ddot{\theta} = u - c\dot{\theta} - \tau_L. \quad (4.1)$$

Under P control,  $u = K_p(r - \theta)$ , this is again a mass-spring-damper,  $J\ddot{\theta} + c\dot{\theta} + K_p\theta = K_p r - \tau_L$ , with damping ratio  $\zeta = c/(2\sqrt{JK_p})$ . Joint friction is small — for  $J = 0.06 \text{ kg m}^2$ ,  $c = 0.05 \text{ N m s}$  and  $K_p = 3 \text{ N m/rad}$ ,  $\zeta \approx 0.06$  — so the arm **rings** for seconds (Figure 6, left). The derivative term adds  $-K_d\dot{\theta}$  to the torque, which is pure extra damping:

$$\zeta = \frac{c + K_d}{2\sqrt{JK_p}}, \quad (4.2)$$

and  $K_d = 0.63$  gives  $\zeta = 0.8$ . Holding a load, PD settles short by  $\tau_L/K_p$  — the offset of Section 3.1 again — and a small I term removes it. All three terms now have a job: P pulls, D damps, I removes the offset.

## 4.1 Derivative kick

Taken literally, (3.1) differentiates the error. When the setpoint jumps, so does the error, by  $\Delta e$  in one sample, and the D term fires a spike of  $K_d \Delta e / \Delta t$ : for the elbow at 1 kHz,  $0.63 \times 1 / 0.001 \approx 630$  N m for one millisecond — enough to saturate the motor and jolt the forearm. Practical controllers therefore differentiate the **measurement** instead, using  $-K_d \dot{y}$ . With a constant setpoint  $\dot{e} = -\dot{y}$ , so nothing else changes; only the kick disappears.

## 4.2 The derivative amplifies noise

Part 1 (§2) showed that a finite difference divides by  $h$ , so every error in the samples is magnified by  $1/h$ . A controller is exactly that situation. With an encoder noise of  $\sigma = 0.002$  rad (about  $0.1^\circ$ ) and  $\Delta t = 1$  ms, the difference  $(y_k - y_{k-1}) / \Delta t$  carries noise of  $\sigma \sqrt{2} / \Delta t \approx 2.8$  rad/s, and the D term turns it into **torque noise**  $K_d \sigma \sqrt{2} / \Delta t \approx 1.8$  N m. The angle barely notices; the motor chatters, heats and wears (Figure 6, right). Sampling faster makes it worse, not better: at 10 kHz the torque noise is ten times larger.

The standard remedy is to **low-pass filter** the derivative,

$$d_f \leftarrow d_f + \frac{\Delta t}{\tau_f} (d_{\text{raw}} - d_f), \quad (4.3)$$

which is itself an Euler step of  $\tau_f \dot{d}_f = d_{\text{raw}} - d_f$ : a running, exponentially weighted average — an integral — of recent derivatives. A filter constant of  $\tau_f = 20$  ms brings the torque noise below 0.1 N m while changing the step response's peak by about one percent.

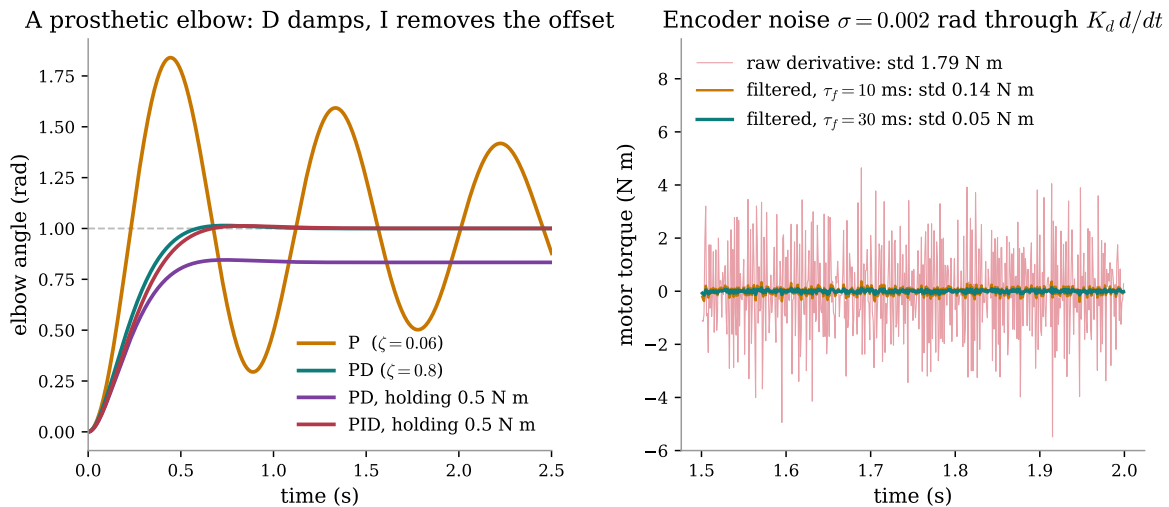


Figure 6: A prosthetic elbow under feedback, simulated with Euler's method at 1 kHz. Left: P alone rings ( $\zeta = 0.06$ ); PD damps it ( $\zeta = 0.8$ ); holding a 0.5 N m load, PD falls short by  $\tau_L / K_p = 0.17$  rad and PID removes the offset. Right: the same PD controller with a 0.002 rad encoder noise, with and without a filtered derivative.

### Key idea

**The two halves of this module, in one controller.** The I term integrates, so it removes constant errors and averages noise away; the D term differentiates, so it adds damping but magnifies noise by  $1/\Delta t$ . Every practical PID implementation — derivative on measurement, a filtered D, anti-windup on the I — is a response to one of those two calculus facts.

**Exercises**

1. Using (4.2) with  $J = 0.06$ ,  $c = 0.05$  and  $K_p = 3$ , find the  $K_d$  that gives critical damping,  $\zeta = 1$ .
2. The elbow holds a 0.5 N m load under PD control with  $K_p = 3$ . Predict the steady-state error, and the value the integrator state  $I$  must reach under PID with  $K_i = 2$  to cancel it.
3. A controller runs at  $\Delta t = 2$  ms with  $K_d = 0.5$  and a sensor noise of  $\sigma = 0.001$  rad. Estimate the torque noise from the D term, and say how it changes if the sampling rate doubles.
4. Explain why a first-order plant such as  $\dot{x} = -ax + bu$  gains little from a D term, while the elbow needs one.

**5 The Lab, and Where This Leads in Géron****Key idea**

**Companion Colab notebook.** Part 2 ships one self-contained notebook:

- `m3_integration_pid` — Riemann sums and the Fundamental Theorem; the trapezoidal and Simpson rules and their error orders, with a drug-exposure AUC; integrating a respiratory flow signal to lung volume, and the drift that sensor bias builds up; Euler's method and its step-size law; and a from-scratch PID controller: P vs PI, the offset the integral must remove, tuning predicted from the characteristic equation, windup and anti-windup, and the D term on a prosthetic elbow with its derivative kick and noise.

It draws on the same biomedical examples as Part 1's notebook, with exercises and solutions. Part 1 — the derivative, the gradient and gradient descent — was taught earlier, with its own companion notebook.

**Where this unlocks the book.** The integral term is the Control unit's PID I; Euler's method is the integrator behind every simulation in the Robotics & Bionics unit, and the same step-size law as Part 1's gradient descent; together with autodiff and Modules 1–2's Python and linear algebra, you have the calculus that Géron's book and the Robotics & Bionics unit run on.

**The one-paragraph summary**

Integration is accumulation and area, inverse to differentiation. Numerically it is a weighted Riemann sum whose error falls like  $h$ ,  $h^2$  or  $h^4$  depending on the rule, and it accumulates a sensor's bias into drift. Simulating a system is integration too: Euler's method is a left Riemann sum of the rate, stable only for  $\Delta t < 2/\lambda$  — gradient descent's learning-rate law in another guise. In a PID controller, the I term's derivative is the error, so it forces the error to zero whenever the loop settles; differentiating the loop once turns it into a mass-spring-damper whose characteristic equation predicts the best  $K_i$ ; and a saturating actuator makes the integral wind up. The D term adds damping and amplifies noise by  $1/\Delta t$ . Differentiate to learn (Part 1); integrate to control (Part 2).